CHAPTER 6 — SOFTWARE METRICS

Numerical Calculations & Solved Problems Pack

Section 1: Halstead's Software Science Calculations

Problem 7.1: Comprehensive Halstead Metric Suite Computation from Complete Token Counts

1. Given: - Unique operators (η_1) = 15 - Unique operands (η_2) = 10 - Total operator occurrences (N_1) = 50 - Total operand occurrences (N_2) = 40 - Stroud neurological constant (S) = $18 \text{ elementary mental discriminations/second}$

2. Required: - Vocabulary (η), Total Length (N), Estimated Length ($\hat{N}$), Program Volume (V), Program Difficulty (D), Program Level (L), Mental Effort (E), Estimated Development Time (T), Language Level (λ), and Estimated Defects ($\hat{B}$).

3. Model & Assumptions: - Model: Maurice Halstead's Software Science Theory (1977). - Assumptions: Standard procedural tokenization; Stroud number $S = 18 \text{ s}^{-1}$; Halstead defect constant $E_0 = 3,000 \text{ discriminations/bug}$.

4. Formula: - $\eta = \eta_1 + \eta_2$ - $N = N_1 + N_2$ - $\hat{N} = \eta_1 \log_2 \eta_1 + \eta_2 \log_2 \eta_2$ - $V = N \log_2 \eta$ - $D = \frac{\eta_1}{2} \times \frac{N_2}{\eta_2}$ - $L = \frac{1}{D}$ - $E = D \times V$ - $T = \frac{E}{18}$ seconds - $\lambda = L^2 \times V$ - $\hat{B} = \frac{V}{3000}$ or $\frac{E^{2/3}}{3000}$

5. Substitution & Working: - $\eta = 15 + 10 = 25$ - $N = 50 + 40 = 90$ - $\log_2 15 = \frac{\ln 15}{\ln 2} \approx 3.90689$; $\log_2 10 = \frac{\ln 10}{\ln 2} \approx 3.32193$

$$\hat{N} = (15 \times 3.90689) + (10 \times 3.32193) = 58.603 + 33.219 = 91.82 \approx 92 \text{ tokens}$$

- $\log_2 25 = \frac{\ln 25}{\ln 2} \approx 4.64386$

$$V = 90 \times 4.64386 = 417.95 \text{ bits}$$

- $D = \frac{15}{2} \times \frac{40}{10} = 7.5 \times 4.0 = 30.0$ - $L = \frac{1}{30.0} \approx 0.0333$ - $E = 30.0 \times 417.95 = 12,538.5 \text{ elementary mental discriminations}$ - $T = \frac{12,538.5}{18} \approx 696.58 \text{ seconds} \approx 11.61 \text{ minutes}$ - $\lambda = (0.03333)^2 \times 417.95 = 0.001111 \times 417.95 \approx 0.464$ - $\hat{B} = \frac{417.95}{3000} \approx 0.139 \text{ defects}$

6. Result: - $\eta = 25$, $N = 90$ tokens, $\hat{N} \approx 92$ tokens - $V = 417.95$ bits, $D = 30.0$, $L = 0.0333$ - $E = 12,538.5 \text{ discriminations}$, $T = 696.58 \text{ s } (11.61 \text{ min})$ - $\lambda \approx 0.464$, $\hat{B} \approx 0.14 \text{ defects}$

7. Check / Interpretation: - The estimated length $\hat{N} \approx 91.82$ is close to the observed $N = 90$ for this example. This numerical agreement alone does not validate the model generally.

8. Source: - *K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 255–262; companion slides 11–23.*

Problem 7.2 (Exercise 6.7): Halstead Metrics with Given Unique Token Counts ($\eta_1 = 20, \eta_2 = 40$) & Mathematical Data-Sufficiency Audit

1. Given: - Number of unique operators (η_1) = 20 - Number of unique operands (η_2) = 40 - Stroud constant (S) = $18\ \mathrm{s}^{-1}$ - Note: Total occurrences (N_1, N_2) are **not** provided in the problem prompt.

2. Required: - Calculate Program Vocabulary (η) and Estimated Program Length ($\hat{N}$). - Conduct a mathematical data-sufficiency analysis determining which Halstead metrics can be definitively calculated versus those requiring unsupplied parameters.

3. Model & Assumptions: - Model: Halstead's Software Science Length Estimator. - Data-Sufficiency Principle: Under strict academic standards, when total occurrences (N_1, N_2) are omitted from the prompt, the actual length $N = N_1 + N_2$ cannot be assumed. We calculate the theoretical length $\hat{N}$ directly from vocabulary and derive proxy estimates where mathematically grounded.

4. Formula: - $\eta = \eta_1 + \eta_2$ - $\hat{N} = \eta_1 \log_2 \eta_1 + \eta_2 \log_2 \eta_2$ - Estimated Volume: $\hat{V} = \hat{N} \log_2 \eta$

5. Substitution & Working: - **Vocabulary (η):**

$$\eta = 20 + 40 = 60$$

- **Logarithm Evaluations:** - $\log_2 20 = \frac{\ln 20}{\ln 2} \approx 4.321928$ - $\log_2 40 = \frac{\ln 40}{\ln 2} \approx 5.321928$ - **Estimated Length ($\hat{N}$):**

$$\hat{N} = (20 \times 4.321928) + (40 \times 5.321928) = 86.43856 + 212.87712 = 299.31568 \approx 299.32 \approx 300\ \text{tokens}$$

- **Estimated Volume Proxy ($\hat{V}$ using $\hat{N}$):** - $\log_2 60 = \frac{\ln 60}{\ln 2} \approx 5.906891$

$$\hat{V} = 299.31568 \times 5.906891 \approx 1,768.03\ \text{bits}$$

- **Audit of Unidentifiable Quantities:** - Program Difficulty $D = \frac{\eta_1}{2} \times \frac{N_2}{\eta_2} = \frac{20}{2} \times \frac{N_2}{40} = 0.25 \times N_2$. Because N_2 (total operand tokens) is unknown, D, L, E, and T cannot be uniquely determined without fabricating an arbitrary value for N_2.

6. Result: - Program Vocabulary (η) = 60 - Estimated Program Length ($\hat{N}$) = $299.32\ \text{tokens}$ (approximately $300\ \text{tokens}$) - Estimated Volume Proxy ($\hat{V}$) $\approx 1,768.03\ \text{bits}$ - Limitations Note: Exact N, V, D, E, T require N_1 and N_2.

7. Check / Interpretation: - Preserving the data-sufficiency boundary prevents ungrounded numerical hallucinations. The problem demonstrates that vocabulary alone is sufficient to predict theoretical program length ($\hat{N} \approx 300$).

8. Source: - *K.K. Aggarwal & Yogesh Singh (3rd ed.), companion slide 102, Exercise 6.7.*

Problem 7.3: Language Level (λ), Program Volume, and Stroud Development Time Conversion

1. Given: - Program Volume (V) = $4,000$ bits - Program Level (L) = 0.05 - Stroud Number (S) = 18 discriminations/second

2. Required: - Program Difficulty (D). - Potential Volume (V^*). - Language Level (λ). - Mental Effort (E). - Estimated Development Time (T) in seconds and hours.

3. Model & Assumptions: - Model: Halstead's Invariant Language Level and Stroud Conversion. - Assumptions: $S = 18\ \text{s}^{-1}$.

4. Formula: - $D = \frac{1}{L}$ - $V^* = L \times V$ - $\lambda = L \times V^* = L^2 \times V$ - $E = \frac{V}{L} = D \times V$ - $T = \frac{E}{S}$

5. Substitution & Working: - $D = \frac{1}{0.05} = 20.0$ - $V^* = 0.05 \times 4000 = 200$ bits - $\lambda = (0.05)^2 \times 4000 = 0.0025 \times 4000 = 10.0$ - Alternatively: $\lambda = L \times V^* = 0.05 \times 200 = 10.0$ - $E = 20.0 \times 4000 = 80,000$ elementary mental discriminations - $T = \frac{80{,}000}{18} \approx 4,444.44$ seconds - $T_{\text{hours}} = \frac{4444.44}{3600} \approx 1.2346$ hours

6. Result: - $D = 20.0$ - $V^* = 200$ bits - $\lambda = 10.0$ - $E = 80,000$ mental discriminations - $T = 4,444.44$ seconds $(1.23$ hours$)$

7. Check / Interpretation: - The computed language-level value is $\lambda = 10.0$. No language classification is inferred here because the inspected course table does not establish an unambiguous mapping for this value.

8. Source: - *K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 259–261; companion slides 18–23.*

Section 2: Data Structure Metrics Calculations

Problem 7.4: Authored Practice Trace — Live Variables, Span, and Module Weakness (WM)

1. Given Source Code Sequence (6 Executable Statements): - S_1: `x = read();` - S_2: `y = read();` - S_3: `z = x + y;` - S_4: `p = z * 2;` - S_5: `print(x);` - S_6: `print(p);`

2. Required: - Statement-by-statement Live Variable sets and counts ($LV(S_i)$). - Average Live Variables ($\overline{LV}$). - Average Life of Variables (γ). - Module Weakness ($WM = \overline{LV} \times \gamma$). - Variable Spans ($SP$) and Average Span Size (SP_{avg}) as supplementary data metrics.

3. Model & Assumptions: - Model: Bieman & Debnath Data Structure Complexity; K.K. Aggarwal Module Weakness formulation. - Assumptions: Count live variables immediately after each statement. A variable is live if defined and referenced later before redefinition. For SP, count only the statements between successive uses; a definition is not a use. Statement count $n = 6$. Total unique variables = 4 (x, y, z, p).

4. Formula: - $\overline{LV} = \frac{\sum_{i=1}^{n} LV(S_i)}{n}$ - $\gamma = \frac{\sum_{i=1}^{n} LV(S_i)}{\text{Total Variables}}$ - $WM = \overline{LV} \times \gamma$ - $SP = b - a - 1$ (span between successive references at statements a and b)

5. Substitution & Working: - **Live Variable Tracing:** - S_1: x is defined; referenced at $S_3, S_5 \rightarrow \{x\} \rightarrow LV(S_1) = 1$ - S_2: y is defined; referenced at S_3; x is active $\rightarrow \{x, y\} \rightarrow LV(S_2) = 2$ - S_3: z is defined; referenced at S_4; x is active; y dies $\rightarrow \{x, z\} \rightarrow LV(S_3) = 2$ - S_4: p is defined; referenced at S_6; x is active; z dies $\rightarrow \{x, p\} \rightarrow LV(S_4) = 2$ - S_5: x has final reference (dies); p is active $\rightarrow \{p\} \rightarrow LV(S_5) = 1$ - S_6: p has final reference (dies) $\rightarrow \emptyset \rightarrow LV(S_6) = 0$ - **Summary of Live Variables:** - Counts: $[1, 2, 2, 2, 1, 0]$ - $LV_{max} = 2$ - $\sum LV = 1 + 2 + 2 + 2 + 1 + 0 = 8$ - $\overline{LV} = \frac{8}{6} \approx 1.33 \text{ variables/statement}$ - **Average Life of Variables (γ):** - Total unique variables = 4 (x, y, z, p) - $\gamma = \frac{8}{4} = 2.0 \text{ statements}$ - **Module Weakness (WM):**

$$WM = \overline{LV} \times \gamma = \frac{8}{6} \times \frac{8}{4} \approx 1.333 \times 2.0 \approx 2.67$$

- **Supplementary Span Calculation:** x is used at S_3 and S_5, so $SP(x) = 5 - 3 - 1 = 1$ intervening statement. The other variables have only one use each, so they add no successive-use span in this example. Thus there is one span, its total is 1, and $SP_{avg} = 1/1 = 1$ statement.

6. Result: - $\overline{LV} = 1.33 \text{ variables/statement}$ - $\gamma = 2.0 \text{ statements}$ - $WM = 2.67$ - $SP_{avg} = 1 \text{ statement}$

7. Check / Interpretation: - $WM = 2.67$ is the value obtained from this constructed trace under the stated counting convention; the metric alone does not establish a universal quality or defect-risk category.

8. Source: - *K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 275–280; companion slides 45–68. Formula source; numerical trace is authored practice.*

Problem 7.5: Program-Wide Weakness (WP) Aggregation Across Modules

1. Given: A software system consists of $m = 4$ modules with the following data metrics: - Module 1: $\overline{LV} = 3.0, \gamma = 4.0 \rightarrow WM_1 = 3.0 \times 4.0 = 12.0$ - Module 2: $\overline{LV} = 1.5, \gamma = 2.0 \rightarrow WM_2 = 1.5 \times 2.0 = 3.0$ - Module 3: $\overline{LV} = 3.5, \gamma = 6.0 \rightarrow WM_3 = 3.5 \times 6.0 = 21.0$ - Module 4: $\overline{LV} = 1.0, \gamma = 1.0 \rightarrow WM_4 = 1.0 \times 1.0 = 1.0$

2. Required: - Program Weakness (WP). - Identification of the primary candidate for refactoring.

3. Model & Assumptions: - Model: Aggarwal's Program Weakness Formulation ($WP = \frac{1}{m} \sum WM_i$).

4. Formula: - $WP = \frac{1}{m} \sum_{i=1}^{m} WM_i$

5. Substitution & Working: - $\sum WM_i = 12.0 + 3.0 + 21.0 + 1.0 = 37.0$ - $WP = \frac{37.0}{4} = 9.25$

6. Result: - Program Weakness (WP) = 9.25 - Module 3 has the largest supplied WM_i and contributes $21/37 \approx 56.8\%$ of the sum used in this exercise. If the goal is to reduce this metric, it is a reasonable module to inspect first; the result alone does not prescribe refactoring.

7. Check / Interpretation: - The stated program formula is the arithmetic mean of the four module values. It gives $WP = 9.25$; Module 3 is highest in this example.

8. Source: - *K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 280; companion slides 63–64.*

Section 3: Information Flow Calculations

Problem 7.6: Information Flow Complexity under Slide 74 Boundary Conventions

1. Given: A modular hierarchy has 3 components: - Component A (Top-level Controller): Raw $fan_in = 0$, calls components B and C ($fan_out = 2$). - Component B (Worker): Called by A ($fan_in = 1$), calls component C ($fan_out = 1$). - Component C (Leaf): Called by A and B ($fan_in = 2$), calls no components (Raw $fan_out = 0$).

2. Required: - Apply Aggarwal's Slide 74 Boundary Rules to determine adjusted fan_in and fan_out. - Calculate Information Flow Complexity $IF = (fan_in \times fan_out)^2$ for each component.

3. Model & Assumptions: - Model: Aggarwal's Slide 74 Procedure. - Boundary Conventions: If raw $fan_in = 0$ for top-level component, set $fan_in = 1$. If raw $fan_out = 0$ for leaf component, set $fan_out = 1$.

4. Formula: - $IF(i) = [fan_in(i) \times fan_out(i)]^2$

5. Substitution & Working: - **Component A:** Raw $fin = 0 \rightarrow$ Adjusted $fin = 1$. $fout = 2$.

$$IF(A) = (1 \times 2)^2 = 2^2 = 4$$

- **Component B:** $fin = 1$, $fout = 1$.

$$IF(B) = (1 \times 1)^2 = 1^2 = 1$$

- **Component C:** $fin = 2$. Raw $fout = 0 \rightarrow$ Adjusted $fout = 1$.

$$IF(C) = (2 \times 1)^2 = 2^2 = 4$$

6. Result: - $IF(A) = 4$, $IF(B) = 1$, $IF(C) = 4$ - Optional arithmetic sum for this exercise (not a separately named standard metric): $4 + 1 + 4 = 9$.

7. Check / Interpretation: - Setting boundary defaults of 1 prevents zero-product collapse while maintaining structural tracking.

8. Source: - *K.K. Aggarwal & Yogesh Singh (3rd ed.), companion slides 71–74.*

Problem 7.7: Henry–Kafura Metric (HKM) with Code Size and Shared Data Structures (Pressman 5e §19.4.2)

1. Given: Module M_{proc} has the following specifications: - Procedural length = $120\ \mathrm{LOC}$ - Subordinate modules called = 3 - Calling higher modules = 2 - Shared global data structures read = 2 - Shared global data structures updated = 1

2. Required: - Total fan_in and total fan_out according to Pressman §19.4.2. - Henry–Kafura Metric (HKM).

3. Model & Assumptions: - Model: Henry and Kafura Structural Complexity Model (Pressman 5e §19.4.2, p. 524). - Rules: $fan_in = \text{calling modules} + \text{data structures read}$;
$fan_out = \text{called modules} + \text{data structures updated}$.

4. Formula: - $fan_in = callers + reads$ - $fan_out = callees + updates$ - $HKM = \text{length} \times [fan_in + fan_out]^2$

5. Substitution & Working: - $fan_in = 2 + 2 = 4$ - $fan_out = 3 + 1 = 4$ - $[fan_in + fan_out] = 4 + 4 = 8$ - $[fan_in + fan_out]^2 = 8^2 = 64$ - $HKM = 120 \times 64 = 7,680$

6. Result: - $fan_in = 4$, $fan_out = 4$ - $HKM = 7,680$

7. Check / Interpretation: - Under this formula, the combined fan-in/fan-out term is squared, so increasing either count can increase the metric substantially. The result is a relative model value, not a calibrated risk probability or threshold.

8. Source: - *Pressman, R., "Software Engineering: A Practitioner's Approach", 5th ed., printed p. 524 / PDF p. 552.*

Section 4: McCabe Cyclomatic Complexity Calculations

Problem 7.8: McCabe Cyclomatic Complexity Computed via All Three Methods

1. Given: A Control Flow Graph (G) representing a validation algorithm with: - Number of nodes (N) = 10 - Number of directed edges (E) = 14 - Number of predicate decision nodes (P_{pred}) = 5 - Planar graph bounded internal regions = 5 - Connected components (P) = 1

2. Required: - Cyclomatic Complexity $V(G)$ computed by: - Method 1: Edge-Node Formula. - Method 2: Predicate Node Formula. - Method 3: Planar Region Formula.

3. Model & Assumptions: - Model: Thomas McCabe's Graph-Theoretic Cyclomatic Complexity (1976). - Assumptions: Planar embedding; single entry and single exit ($P = 1$).

4. Formula: - Method 1: $V(G) = E - N + 2P$ - Method 2: $V(G) = P_{pred} + 1$ - Method 3: $V(G) = \text{Enclosed Regions} + \text{Unbounded Outer Region}$

5. Substitution & Working: - Method 1: $V(G) = 14 - 10 + 2(1) = 4 + 2 = 6$ - Method 2: $V(G) = 5 + 1 = 6$ - Method 3: $V(G) = 5 + 1 = 6$

6. Result: - $V(G) = 6$ (All three methods yield exactly 6).

7. Check / Interpretation: - Under the basis-path method, $V(G) = 6$ gives the number of linearly independent basis paths for the modeled control-flow graph. It does not by itself establish low risk or guarantee full branch coverage.

8. Source: - *K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 422–424; companion slides Ch.8.*

Problem 7.9: Cyclomatic Complexity Decomposition of Compound Decision Predicates

1. Given Source Code Fragment: `c if (a > 0 && b < 10) { process_primary(); } else if (c == 0 || d != 5) { process_secondary(); } else { process_default(); }`

2. Required: - Decompose compound Boolean conditions into elementary predicate nodes. - Calculate total Cyclomatic Complexity $V(G)$.

3. Model & Assumptions: - Model: McCabe's Compound Condition Rule. - Rule: A compound decision containing k simple Boolean conditions separated by logical operators (`&&`, `||`) contributes k individual predicate nodes to the control flow graph.

4. Formula: - $V(G) = P_{\text{simple_pred}} + 1$

5. Substitution & Working: - Condition 1: `(a > 0 && b < 10)` contains 2 simple conditions (`a > 0`, `b < 10`) $\rightarrow 2$ decision nodes. - Condition 2: `(c == 0 || d != 5)` contains 2 simple conditions (`c == 0`, `d != 5`) $\rightarrow 2$ decision nodes. - Total simple decision nodes $P_{\text{simple_pred}} = 2 + 2 = 4$. - $V(G) = 4 + 1 = 5$.

6. Result: - $V(G) = 5$

7. Check / Interpretation: - Treating compound conditions as single nodes would produce an incorrect $V(G) = 3$, failing to account for the short-circuit evaluation branches required during testing.

8. Source: - *K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 424.*

Section 5: Size & Function Point Calculations

Problem 7.10 (Exercise 6.11): Reassess a Payroll Function-Point Count After Scope Expansion

1. Given: - Chapter 4 Exercise 4.10 describes the baseline payroll system; its published problem gives functional requirements but does not provide all DET/FTR complexity data needed for a unique exact count. - Exercise 6.11 adds storing printed cheques for later use and producing a comparison report against the previous month.

2. Required: Explain how the added requirements affect the function-point count and give a reproducible illustrative calculation.

3. Model & Assumptions: Use Albrecht-style unadjusted function points. For illustration only, classify the baseline as two average ILFs, one average EI, and two average EOs; classify the added historical file as one average ILF and the two added outputs as average EOs. These complexity ratings are assumptions because DET/FTR counts are not supplied. Do not treat the result as a unique textbook answer.

4. Formula: $UFP = \sum(\text{count} \times \text{weight})$. Average weights used here: ILF=10, EI=4, EO=5.

5. Substitution & Working: - Illustrative baseline: $UFP_{old} = 2(10) + 1(4) + 2(5) = 34$. - Illustrative additions: one historical ILF and two outputs give $\Delta UFP = 1(10) + 2(5) = 20$. - Illustrative expanded count: $UFP_{new} = 34 + 20 = 54$. - Storing the historical file adds maintained internal data; retrieving/printing it and producing a comparison report add user-visible processing. Exact function types, complexity weights, and any adjusted FP value depend on boundary, DET/FTR counts, and the counting method.

6. Result: Under the stated average-weight assumptions, the illustrative count rises from 34 to 54 UFP. This is not a uniquely determined exact count from the exercise statement alone.

7. Check / Interpretation: The difference is caused by expanded retained data and reporting requirements. “Interfacing with itself” should not automatically be counted as a separate external application; establish the application boundary and avoid double-counting the historical file or report.

8. Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), Exercise 6.11, printed p. 306 / companion slide 103; Chapter 4 Exercises 4.10–4.11, printed pp. 201–202. Weights from the supplied Chapter 4 material; complexity assignments above are illustrative assumptions.

Problem 7.11: Complete Albrecht Function Point Computation with General System Characteristics

1. Given Information Domain Counts & Complexity Ratings: - External Inputs (EI): 10 Low (Weight = 3), 5 Average (Weight = 4) - External Outputs (EO): 6 Average (Weight = 5), 2 High (Weight = 7) - External Inquiries (EQ): 8 Low (Weight = 3) - Internal Logical Files (ILF): 4 Average (Weight = 10) - External Interface Files (EIF): 2 High (Weight = 10) - 14 General System Characteristics: Evaluated with sum of degrees of influence $\sum F_i = 42$.

2. Required: - Unadjusted Function Points (UFP). - Value Adjustment Factor (VAF). - Final Adjusted Function Points (FP).

3. Model & Assumptions: - Model: Albrecht IFPUG Standard Model.

4. Formula: - $UFP = \sum(Count \times Weight)$ - $VAF = 0.65 + 0.01 \sum_{i=1}^{14} F_i$ - $FP = UFP \times VAF$

5. Substitution & Working: - $UFP(EI) = (10 \times 3) + (5 \times 4) = 30 + 20 = 50$ - $UFP(EO) = (6 \times 5) + (2 \times 7) = 30 + 14 = 44$ - $UFP(EQ) = (8 \times 3) = 24$ - $UFP(ILF) = (4 \times 10) = 40$ - $UFP(EIF) = (2 \times 10) = 20$ -

$UFP = 50 + 44 + 24 + 40 + 20 = 178$ UFP - $VAF = 0.65 + 0.01(42) = 0.65 + 0.42 = 1.07$ - $FP = 178 \times 1.07 = 190.46$ FP

6. Result: - $UFP = 178$ - $VAF = 1.07$ - $FP = 190.46$ FP

7. Check / Interpretation: - Under the stated formula, $VAF = 1.07$ multiplies the unadjusted count by 1.07, a 7% increase. No qualitative label for the environment follows from this value alone.

8. Source: - *K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 142–146.*

Section 6: Object-Oriented, UCP & Quality Calculations

Problem 7.12: Chidamber & Kemerer (CK) Metric Suite Derived on a 3-Class Design

1. Given Class Design Specification: - **Class `Account` (Root Class):** - Instance variables: `balance`, `accNum`. - Methods: `deposit()` (uses `balance`), `withdraw()` (uses `balance`), `getAccNum()` (uses `accNum`). - Collaborations: Calls no external classes. - **Class `SavingsAccount` (Subclass of `Account`):** - Instance variables: `interestRate`. - Methods: `addInterest()` (uses `interestRate`, calls `deposit()`), `calculateTax()` (uses `interestRate`, calls `getAccNum()`). - **Class `AuditReporter` (Standalone Class):** - Instance variables: `logCount`. - Methods: `logAccount(SavingsAccount sa)` (calls `sa.getAccNum()`, uses `logCount`).

2. Required: - Compute WMC, DIT, NOC, CBO, RFC, and LCOM for Class `Account`.

3. Model & Assumptions: - Model: Chidamber & Kemerer (CK) Suite (1994). Method weights $c_i = 1$.

4. Formula: - $WMC = \sum c_i$ - $DIT = \max$ path to root - $NOC =$ direct subclasses - $CBO =$ distinct coupled classes - $RFC = |M \cup \bigcup R_m|$ - $LCOM = P - Q$ (if $P > Q$, else 0)

5. Substitution & Working for Class `Account`: - **WMC:** 3 methods (`deposit`, `withdraw`, `getAccNum`) with unit weights $\rightarrow WMC = 3$. - **DIT:** `Account` is the root class $\rightarrow DIT = 0$. - **NOC:** Directly inherited by `SavingsAccount` $\rightarrow NOC = 1$. - **CBO:** `Account` does not invoke external classes $\rightarrow CBO = 0$. - **RFC:** Internal methods $M = \{\text{deposit}, \text{withdraw}, \text{getAccNum}\}$. Invokes no external methods $\rightarrow RFC = 3$. - **LCOM:** - Methods: $m_1 = \text{deposit}$ (`balance`), $m_2 = \text{withdraw}$ (`balance`), $m_3 = \text{getAccNum}$ (`accNum`). - Method pairs (total $\binom{3}{2} = 3$ pairs): - (m_1, m_2): Share `balance` $\rightarrow$ Share (Q). - (m_1, m_3): No shared variables $\rightarrow$ Disjoint (P). - (m_2, m_3): No shared variables $\rightarrow$ Disjoint (P). - $P = 2$ disjoint pairs; $Q = 1$ sharing pair. - $LCOM = P - Q = 2 - 1 = 1$.

6. Result for Class `Account`: - $WMC = 3$, $DIT = 0$, $NOC = 1$, $CBO = 0$, $RFC = 3$, $LCOM = 1$.

7. Check / Interpretation: - The computed values are $LCOM = 1$ and $CBO = 0$ under the stated counting rules. These values alone do not establish that the class is acceptable or reusable.

8. Source: - *K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 288–294.*

Problem 7.13: Calculate Unadjusted Use Case Point Components from the Supplied Course Weights

1. Given: - Actors: 2 simple, 3 average, and 4 complex. - Use cases: 5 simple, 8 average, and 3 complex. - Course weights: simple/average/complex actors = 1/2/3; simple/average/complex use cases = 5/10/15.

2. Required: Calculate UAW, $UUCW$, and $UUCP$.

3. Model & Assumptions: Use only the unadjusted actor and use-case weights given in the supplied course slides. The approved local sources inspected for this chapter do not supply verified technical/environmental adjustment-factor rules or an effort-per-UCP conversion, so no adjusted UCP or effort estimate is calculated.

4. Formula: - $UAW = \sum(\text{actor count} \times \text{actor weight})$ - $UUCW = \sum(\text{use-case count} \times \text{use-case weight})$ - $UUCP = UAW + UUCW$

5. Substitution & Working: - $UAW = (2 \times 1) + (3 \times 2) + (4 \times 3) = 2 + 6 + 12 = 20$. - $UUCW = (5 \times 5) + (8 \times 10) + (3 \times 15) = 25 + 80 + 45 = 150$. - $UUCP = 20 + 150 = 170$.

6. Result: $UAW = 20$, $UUCW = 150$, and $UUCP = 170$ unadjusted points.

7. Check / Interpretation: This result stops before TCF, ECF, adjusted UCP, or person-hours because those rules are not established by the inspected approved course-source set.

8. Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 295–296; companion slides 89–90.

Problem 7.14: Defect Removal Efficiency (DRE) and Defect Density Calculations

1. Given: An e-commerce system of size $40\ \text{KLOC}$ undergoes rigorous multi-phase quality assurance: - Defects discovered and resolved internally during requirements, design, code reviews, and integration testing (E) = $380\ \text{defects}$ - Defects reported by customers during the first 6 months of production operation (D) = $20\ \text{defects}$

2. Required: - Defect Removal Efficiency (DRE). - Internal Defect Density ($\text{Defects}/KLOC$). - Released (Post-Delivery) Defect Density ($\text{Defects}/KLOC$).

3. Model & Assumptions: - Model: Standard Quality Metrics Framework (Pressman 5e).

4. Formula: - $DRE = \frac{E}{E+D}$ - $\text{Internal Defect Density} = \frac{E}{KLOC}$ - $\text{Released Defect Density} = \frac{D}{KLOC}$

5. Substitution & Working: - $DRE = \frac{380}{380+20} = \frac{380}{400} = 0.95 = 95.0\%$ - $\text{Internal Defect Density} = \frac{380}{40} = 9.5\ \text{defects/KLOC}$ - $\text{Released Defect Density} = \frac{20}{40} = 0.5\ \text{defects/KLOC}$

6. Result: - $DRE = 95.0\%$ - Internal Defect Density = $9.5\ \text{defects/KLOC}$ - Released Defect Density = $0.5\ \text{defects/KLOC}$

7. Check / Interpretation: - These are the calculated values for the supplied counts and size. They cannot be classified as good or poor without a comparable baseline, consistent counting rules, and an observation period.

8. Source: - *Pressman, R., 5th ed., printed p. 98 / PDF p. 126; companion slide 10.*

Problem 7.15: Software Productivity and Phase Containment Effectiveness (PCE)

1. Given: A development project delivered $500\ \text{Function Points}$ using $25\ \text{Person-Months (PM)}$ of effort: - Requirements phase: Injected 100 defects. Inspected and caught 75 defects before design. - Design phase: Injected 60 defects. Inspected and caught 48 design defects before coding.

2. Required: - Project Productivity in FP/PM. - Phase Containment Effectiveness for Requirements (PCE_{req}) and Design (PCE_{des}).

3. Model & Assumptions: - Productivity is computed as delivered size divided by effort. For this exercise only, define PCE_i as the fraction of phase-i defects caught in that phase; use the supplied injected-defect counts as the denominator.

4. Formula: - $\text{Productivity} = \frac{\text{Delivered Size}}{\text{Effort}}$ - Exercise-specific operational definition: $PCE_i = \frac{D_i^{\text{caught}}}{D_i^{\text{injected}}} \times 100\%$

5. Substitution & Working: - $\text{Productivity} = \frac{500\ \text{FP}}{25\ \text{PM}} = 20.0\ \text{FP/PM}$ - $PCE_{req} = \frac{75}{100} \times 100\% = 75.0\%$ - $PCE_{des} = \frac{48}{60} \times 100\% = 80.0\%$

6. Result: - Productivity = $20.0\ \text{Function Points/Person-Month}$ - $PCE_{req} = 75.0\%$ - $PCE_{des} = 80.0\%$

7. Check / Interpretation: - Under the stated counts, 25 of the 100 requirements defects were not caught within requirements, and 12 of the 60 design defects were not caught within design. The values describe this example; they do not establish a benchmark or identify a cause.

8. Source: - *Pressman, R., 5th ed., printed pp. 92–95 (productivity context); Mall, 4th ed., PDF pp. 84–85 (phase-containment concept). The PCE equation is the exercise-specific definition above.*

- Compute Halstead's Volume and Difficulty from $\eta_1 = 15, \eta_2 = 10, N_1 = 50, N_2 = 40$.
- Explain why Exercise 6.7 ($\eta_1 = 20, \eta_2 = 40$) can determine $\hat{N}$ but not actual N or D.
- Apply the 3 computation methods for McCabe Cyclomatic Complexity on a 10-node, 14-edge graph.
- Re-derive Henry–Kafura metric for a module of 120 LOC with $fin = 4, fout = 4$.
- Calculate UCP from raw actor and use-case counts and convert to person-hours.